Assignment # 03

Name: Eman Faris

Roll No.: 23i-5572

Section: FT-6B

Part B:

Technical Report

1. Problem Definition

What your assigned system solves

The system solves the challenge of identifying high-risk financial activities in real-time. The system must distinguish between legitimate user behavior and potential fraud which ranges from identity theft to account takeovers. By applying Risk Evaluation logic, the system flags suspicious transactions so they can be reviewed or blocked before the funds are cleared.

2. System Architecture

Diagram or description of MERN stack flow

The system follows the standard MERN (MongoDB, Express, React, Node.js) flow:

- **Frontend (React):** A dashboard where admins can view a list of transactions and their associated "Risk Score."
- **Backend (Node.js/Express):** RESTful API that receives transaction data and runs the Risk Evaluation algorithm.
- **Database (MongoDB):** Stores all transaction logs and the calculated risk results in a single collection.

3. Database Design

Schema diagrams, field definitions, relationship justification

The requirement for my assigned system is **Basic (1 collection)**, that is why I used a simple "Transactions" collection. This collection will store both the raw data and the evaluation output.

Collection: `transactions`

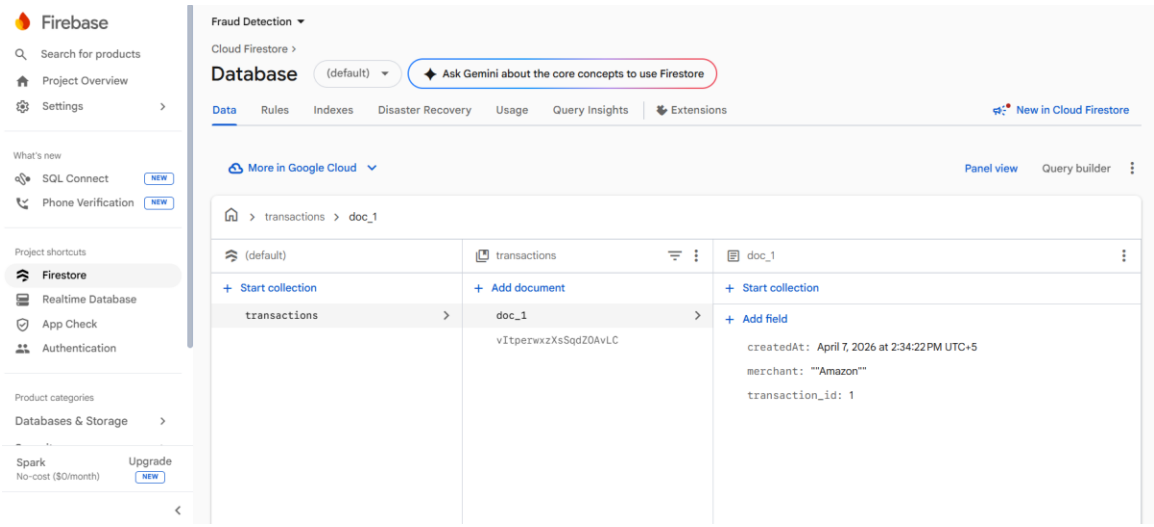
- `transactionID`: String (Unique ID)
- `amount`: Number (The monetary value)
- `location`: String (IP address or City)
- `timestamp`: Date
- `riskScore`: Number (0–100)
- `status`: String (such as "Cleared", "Flagged", "Blocked")

Schema Diagram:

Transactions
transaction_id amount location timestamp risk_score status

Relationship Justification: No external relationships are required for this level, ensuring fast read/write speeds for high-velocity transaction data.

Firebase Implementation:



4. Core Logic Explanation

Step-by-step explanation of your FinTech logic

The "Risk Evaluation" logic is the brain of my assigned app. It follows these steps:

1. **Input:** Receives a transaction object.
2. **Velocity Check:** Checks if the amount exceeds a specific threshold (for instance; > $10,000).
3. **Geographic Check:** Compares the current location with the user's "Home" location (logic-based).
4. **Scoring:** Start at 0.
    - Add 50 points if the amount is unusually high.
    - Add 30 points if the location is foreign.
5. **Output:** If the score is > 70, sets `status` to "Blocked.

5. Query Explanation

Explain your two meaningful queries with sample data

Following are the two primary queries required to make the dashboard functional:

- **Query 1 (High-Risk Retrieval):** Fetches all transactions where the risk score is high.
    - *Sample Data:* `db.transactions.find({ riskScore: { $gt: 80 } })`
- **Query 2 (Aggregated Fraud Volume):** Calculates the total amount of money currently "Flagged" to see the financial impact.

- *Sample Data:* `db.transactions.aggregate([{ $match: { status: "Flagged" } }, { $group: { _id: null, total: { $sum: "$amount" } } }])`

6. Security Analysis

What you implemented and why

For my assigned app, I implemented:

- **Input Sanitization:** Using libraries like `express-validator` to prevent NoSQL injection.
- **JWT (JSON Web Tokens):** To ensure only authorized admins can access the fraud dashboard.
- **Data Encryption:** Encrypting sensitive fields (like partial account numbers if added) at rest using MongoDB's encryption or `bcrypt` for passwords.

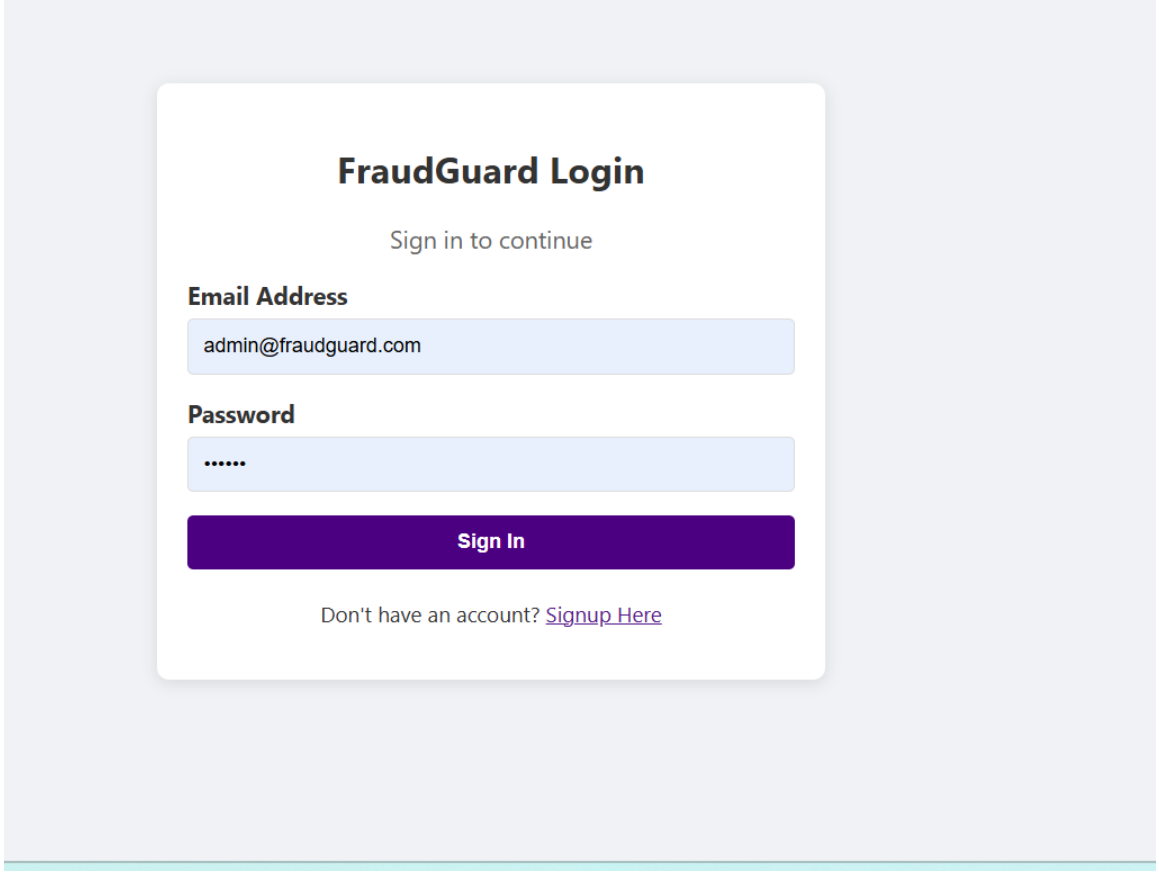
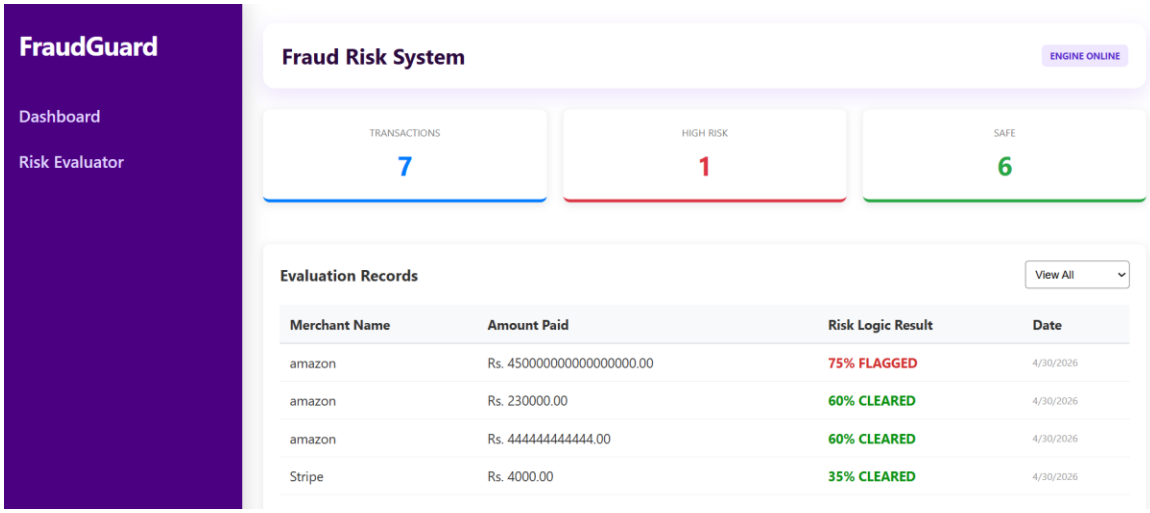
7. Scalability Discussion

What would break at 10,000 users and how to fix it?

The Bottleneck: At 10,000 users, a single-collection MongoDB instance might experience **Write Locking**. If thousands of transactions hit the database simultaneously, the "Risk Evaluation" logic (which happens in Node.js) might lag, causing a backup.

The Fix:

1. **Horizontal Scaling:** Use a Load Balancer to distribute traffic across multiple Express server instances.
2. **Indexing:** Create an index on the `riskScore` and `status` fields to keep queries fast.
3. **Caching:** Use Redis to store frequently accessed risk rules so the server doesn't have to calculate everything from scratch for every request.

UI:

On vscode's terminal, run the app by npm start